Figure 7: Logged in page

Figure 8: New user page

```
selection = username=? AND password=?
selectionArgs = uname,pass
```

In our query, we selected the table column "*_ID*" from the table *SAKET_TABLE_NAME*, which is the table I have created to store the user credentials. The *selection* parameter has the *WHERE* clause (the condition of the query that needs to be verified with the database); *selectionArgs* has the values of the place-holders in the *selection* parameter.

This query returns a *CURSOR*—a collection of data returned by the query. *CURSOR* is a database construct used to hold values when a query returns more than one row of data. It needs to be opened first, before you can issue a *FETCH* command to get the contents of the cursor, and last of all, you need to close the cursor. The task of managing a cursor is done by the API *startManagingCursor(cursorname)*. This opens and closes the cursor.

After getting the cursor, we can get the data from the cursor using its built-in methods, such as:

- *getCount()*—the count of rows returned by the query
- *getXXX(int columnIndex)*—based on the data type of the column, you can retrieve the data. For example, *getInt(int columnIndex)*, *getDouble*, *getString*, etc.
- *getColumnIndex(String columnName)*: Returns you the column index for the named column.

Using these API methods, we can retrieve the data. In this scenario, usernames are assumed to be unique; if the username exists in the table, then the *getCount()* method should return only one row. If the row count is 0, that means that there is no such user record; then, we Toast the user with the appropriate message. Now, when the users' credentials are verified, they are taken to a 'logged-in' page, as shown below.

NewUserActivity is the activity that registers a *new user*. In this activity, we demonstrate how to insert a row of data in the database. When users click on the New User button, they are taken to the *New User* page. In this page, the users need to enter the new username, password, and email address. When that is done, they need to click the *Register* button, as shown in Figure 8.

The *Register* button invokes a method *addEntry()* in the *NewUserActivity* class, which adds the entries of the users in the database, thus registering them.

```
public void addEntry(String uname, String pass, String email){
      SQLiteDatabase db = myDb.getWritableDatabase();
      ContentValues values = new ContentValues();
      values.put("username", uname);
      values.put("password", pass);
      values.put("email", email);

      try{
        db.insert(DbHelper.SAKET_TABLE_NAME, null, values);
        Toast.makeText(getApplicationContext(), "User
Registered",                Toast.LENGTH_SHORT).show();
      } catch(Exception e){
        e.printStackTrace();
      }
}
```

Here, we open the database for writing, by invoking the *getWritableDatabase()* method. In Android, the values that need to be inserted are entered using *ContentValues*. An instance of this class is passed in the *INSERT* statement, whose syntax is as follows:

```
public long insert (String table, String nullColumnHack,
ContentValues values)
```

Here, the parameters are as follows:

- *table*: table name
- *nullColumnHack*: SQL doesn't allow inserting a completely empty row, so if *initialValues* is empty, this column will explicitly be assigned a *NULL* value.
- *ContentValues*: Holds the values that the *INSERT* statement seeks to insert in the database.

*DBHe*lper is the class that creates the database, and it can also be configured to upgrade the database (if, in future, the developer changes the database design in later versions of the application).

```
public class DbHelper extends SQLiteOpenHelper {
      private static final String DATABASE_NAME = "saket.db";
```